DATA PRODUCT CASE STUDY

PRIME: one governed data layer for a multi-tenant product

Then the same approach, live on City of Calgary transit data.

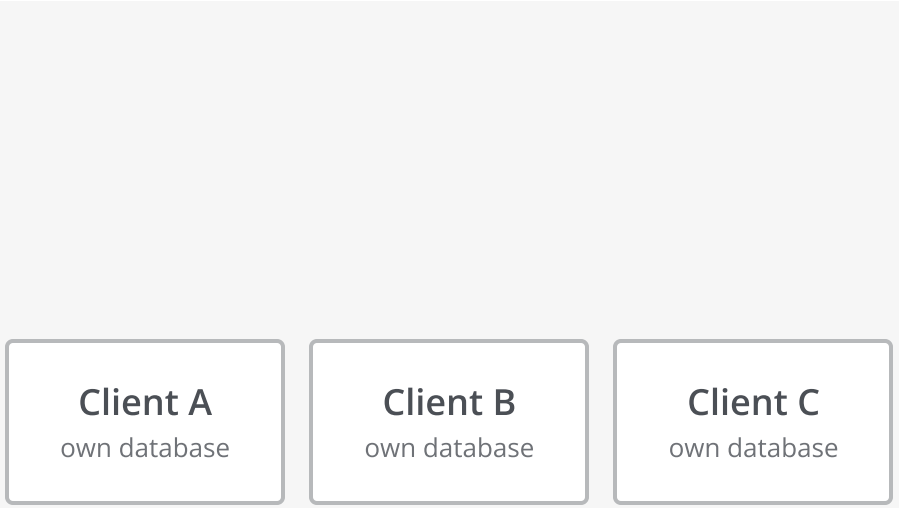
Cody Chandler
Data and Business Analyst interview · The City of Calgary

I'll answer them in order

1. What business problem was I solving?
2. What is the data product?
3. How did I approach it?
4. What challenges came up?
5. What am I most proud of?
6. What would I improve with more time?

With a 5-minute live demo on City of Calgary data after question 3.

What business problem was I solving?



The problem

A multi-tenant product kept each client's data in its own database, so internal reporting was stale (up to 7 days old), fragmented and unowned.

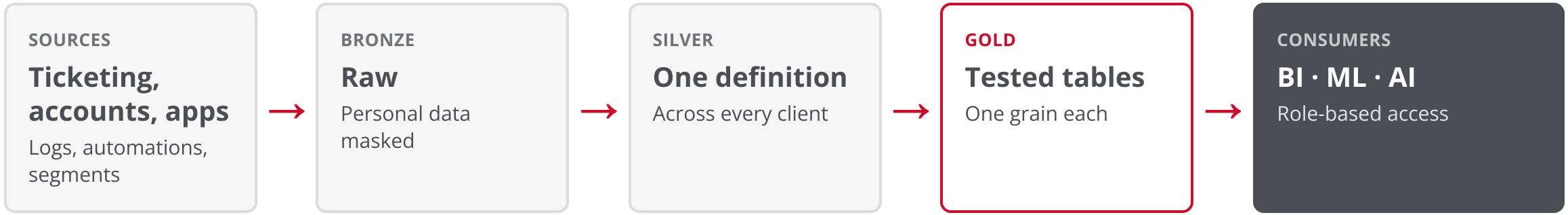
a Why it mattered

Teams and clients worked from different numbers, and every cross-client question needed engineering.

b Who it affected

Customer Success · Product · Clients · Analysts · AI assistants

What is the data product?



a What it does

Consolidates every client into one governed, tested layer, refreshed daily, with a data contract: owners, targets and incident tiers.

b How people use it

- BI dashboards
- Client data sharing
- CS platform
- Plain-English questions

C What value it delivered

~~7 days~~

< 12 hrs

Data freshness

100+

Tested models, replacing a database per client

30 min

To flag a missed refresh

Self-serve

Routine Customer Success questions

When the dashboard, the client export and the AI give the same number, people stop checking and start deciding.

How did I approach it?

Problem → Needs → Strategy → Plan → Build → **Review and iterate**

a **Sourcing and preparing the data**

Ticketing, accounts, app logs, automations, segments and reference files: landed raw, personal data masked, then modelled into one definition and tested.

b **Methods and tools**

Snowflake · dbt · Airflow · AWS · Medallion layers · Data contract · Semantic layer · LLMs

The same approach, live on City of Calgary data

1. Dashboard: riders back to 95% of 2019
2. The catch: a 43% jump that wasn't real
3. Predict: a live XGBoost model
4. Ask the data: plain English to SQL

Airflow · dbt · DuckDB · 27 tests · XGBoost · Claude · codychandler.ca/data-demos/calgary-transit

What challenges came up, and how did I handle them?

Type	Challenge	How I solved it
Technical	Every client system had its own shape.	One dimensional model all clients map into.
Analytical	The wrong table, or a changed definition, quietly misleads.	One grain per table, and governed fixes like Lost Property.
Interested parties	Nobody owned freshness or accuracy.	A data contract with named owners, targets and tiers.

What am I most proud of?

AI answers that match the dashboard, and show their work.

1. **Built on the governed layer**
   One documented grain per table, so the agent can't double count.

2. **Shows its SQL**
   Anyone can check the number it gives.

3. **Same tables for BI, ML and AI**
   Governance stops being overhead and becomes the reason people can self-serve.

What would I improve with more time?

PRIME for clients

Take the internal CS bot to clients as Stellar Insights.

On-time performance

Add the City's real-time feeds to measure reliability by route.

Owner sign-off

Agree targets and definitions with the people who run the service.

The same work at the City: find where numbers disagree, agree one definition with the owners, and make it self-serve.

Thank you. Questions?

Live demo	**codychandler.ca/data-demos/calgary-transit**
Code	**github.com/frontier-platforms/cody-portfolio**